

软件工程基础

第 2 章——软件过程

Software Engineering Fundamentals – Chapter 2: Software Process

详细学习笔记

2026 年 5 月 28 日

本章核心内容总览

- 软件过程（**Software Process**）的基本概念与 GB/T8566 标准体系
- 传统软件过程模型：瀑布模型、原型模型、RAD 模型、增量模型、螺旋模型、统一过程（RUP）
- 敏捷软件开发（**Agile Software Development**）：敏捷宣言、12 条敏捷原则
- 极限编程（**XP**）：价值观、原则、实践、过程（规划/迭代/测试/发布）
- **Scrum**：角色、制品、会议、燃尽图
- 其他敏捷方法：精益软件开发、特征驱动开发（FDD）
- 敏捷项目管理工具：PingCode、Gitee

目录

1 软件过程概述 (<i>Software Process Overview</i>)	5
1.1 软件过程的基本概念	5
1.2 软件生存期过程模型	5
1.3 GB/T8566 国家标准体系	5
1.3.1 GB/T8566-1995 (旧版)	5
1.3.2 GB/T8566-2007 (新版)	5
1.4 软件生存周期模型	6
2 传统软件过程模型 (<i>Traditional Software Process Models</i>)	7
2.1 瀑布模型 (<i>Waterfall Model</i>)	7
2.1.1 模型概述	7
2.1.2 瀑布模型的特点	7
2.1.3 瀑布模型的不足	7
2.2 原型模型 (<i>Prototype Model</i>)	7
2.2.1 模型概述	7
2.2.2 快速原型模型的特点	8
2.2.3 原型模型的不足	8
2.3 快速应用开发模型 (<i>RAD Model — Rapid Application Development</i>)	8
2.3.1 模型概述	8
2.3.2 RAD 模型的特点	9
2.3.3 RAD 模型的不足	9
2.4 增量模型 (<i>Incremental Model</i>)	9
2.4.1 模型概述	9
2.4.2 增量模型的特点	9
2.4.3 增量模型的不足	10
2.5 螺旋模型 (<i>Spiral Model</i>)	10
2.5.1 模型概述	10
2.5.2 螺旋模型的特点	10
2.5.3 螺旋模型的不足	11
2.6 统一过程模型 (<i>Unified Process — UP / RUP</i>)	11
2.6.1 模型概述	11
2.6.2 四个开发阶段	11
2.6.3 核心工作流 (<i>Core Workflows</i>)	12
2.6.4 RUP 的三大特点	12
2.6.5 RUP 的不足	12
3 传统过程模型对比 (<i>Comparison of Traditional Models</i>)	13

4 敏捷软件开发 (<i>Agile Software Development</i>)	14
4.1 敏捷运动的背景	14
4.2 敏捷联盟与敏捷宣言 (<i>Agile Alliance & Agile Manifesto</i>)	14
4.2.1 敏捷的定义	14
4.3 12 条敏捷原则	14
4.3.1 敏捷原则的分类理解	15
5 极限编程 (<i>Extreme Programming — XP</i>)	16
5.1 XP 概述	16
5.2 XP 与瀑布模型及迭代模型的区别	16
5.3 XP 的五大价值观 (<i>Five XP Values</i>)	16
5.4 XP 的实践原则	16
5.5 XP 核心实践体系	16
5.6 XP 开发过程详解	17
5.6.1 规划阶段 (Planning Phase)	17
5.6.2 迭代阶段 (Iteration Phase)	18
5.6.3 测试阶段 (Testing Phase)	20
5.6.4 发布阶段 (Release Phase)	20
6 Scrum 敏捷软件开发过程	22
6.1 Scrum 概述	22
6.2 Scrum 团队的三种角色	22
6.3 开发团队的特点	22
6.4 PDCA 循环 (戴明环) 与冲刺的关系	22
6.5 Scrum 制品 (<i>Scrum Artifacts</i>)	23
6.5.1 产品待办事项表 (<i>Product Backlog</i>)	23
6.5.2 冲刺待办事项表 (<i>Sprint Backlog</i>)	23
6.5.3 冲刺燃尽图 (<i>Sprint Burndown Chart</i>)	23
6.5.4 发布燃尽图 (Release Burndown Chart)	24
6.6 Scrum 会议 (<i>Scrum Meetings</i>)	24
6.6.1 冲刺计划会 (<i>Sprint Planning Meeting</i>)	24
6.6.2 每日站立会议 (<i>Daily Stand-up</i>)	24
6.6.3 冲刺评审会议 (<i>Sprint Review Meeting</i>)	24
6.6.4 冲刺反思会 (<i>Sprint Retrospective</i>)	25
7 其他敏捷方法 (<i>Other Agile Methods</i>)	26
7.1 敏捷方法的实际使用统计	26
7.2 精益软件开发 (<i>Lean Software Development</i>)	26
7.3 特征驱动软件开发 (<i>Feature-Driven Development — FDD</i>)	26
7.3.1 什么是特征	26
7.3.2 FDD 的八个最佳实践	26

7.3.3 FDD 的五个子过程	27
8 敏捷项目管理工具 (<i>Agile Project Management Tools</i>)	28
8.1 PingCode	28
8.2 Gitee 项目管理	28
9 本章核心概念总结	29
9.1 关键术语中英对照表	29
9.2 传统模型与敏捷方法的对比	30

1 软件过程概述 (*Software Process Overview*)

1.1 软件过程的基本概念

软件产品生产由一组相互有机联系起来的活动来完成，这些相互有机联系的活动便构成软件过程（**Software Process**）。软件过程的三个核心要素是：

- (1) 软件开发（**Development**）——依据需求进行软件的设计与编码
- (2) 软件管理（**Management**）——对软件项目的计划、组织、协调和控制
- (3) 软件质量（**Quality**）——决定软件产品的质量保障

以上三者共同决定了软件过程模型（**Software Process Model**）的选择。[p.4]

1.2 软件生存期过程模型

软件生存期过程模型（**Software Life Cycle Process Model**）是软件生存期各项活动或任务或子过程的有机结合，以满足特定项目需要。[p.6]

软件生存期过程规定了获取、供应、开发、操作和维护软件时要实施的过程、活动和任务。其目的是为各种人员提供一个公共的框架，以便可以使用“相同的语言”在自己的环境中创作和管理软件。

关键认识：软件生存期过程没有规定一个特定的生存周期模型，各软件开发机构可视其项目的需要选择一种软件生存周期模型，并将软件生存期过程所含过程、活动和任务映射到选定的软件生存周期模型中。[p.6]

1.3 GB/T8566 国家标准体系

中国国家标准 GB/T8566 对软件生存期活动进行了规范化描述。

1.3.1 GB/T8566-1995（旧版）

将软件生存期中的活动分成 **7 个基本的、主要的生存期过程**：[p.8]

- 1) 获取过程 (*Acquisition Process*)
- 2) 供应过程 (*Supply Process*)
- 3) 管理过程 (*Management Process*)
- 4) 开发过程 (*Development Process*)
- 5) 操作过程 (*Operation Process*)
- 6) 维护过程 (*Maintenance Process*)
- 7) 支持过程 (*Supporting Process*)

1.3.2 GB/T8566-2007（新版）

将软件生存期中可能执行的活动分为三类共 **21 个过程**：[p.9-12]

- **五个基本过程：**获取过程、供应过程、开发过程、运作过程、维护过程
- **九个支持过程：**文档编制、配置管理、质量保证、验证、确认、联合评审、审核、问题解决、易用性

- **七个组织过程：**管理过程、基础设施过程、改进过程、人力资源过程、资产管理过程、复用过程、领域工程过程

1.4 软件生存周期模型

软件生存周期模型 (Software Development Life Cycle Model) 亦称**软件开发模型**，描述了从软件项目需求定义开始，到开发成功后投入使用，在使用中不断增补修订，直到停止使用，这一期间各种活动如何执行的模型。[p.17]

软件开发机构应该综合项目和应用的性质、将要使用的方法和工具等，选择其中合适的模型或构建，并将软件生存期过程映射到选定的模型中进行软件开发和维护。[p.17]

2 传统软件过程模型 (*Traditional Software Process Models*)

本章介绍六种主要的传统软件过程模型，每种模型都有其独特的结构、特点、优点和不足。

2.1 瀑布模型 (*Waterfall Model*)

2.1.1 模型概述

瀑布模型 (**Waterfall Model**), 也称为传统的生命周期模型, 最初由温斯顿·罗伊斯 (Winston Royce) 在 1970 年提出。[p.19]

瀑布模型将软件开发过程划分为若干个**顺序执行**的阶段, 每个阶段有明确的输入和输出, 前一阶段的输出是后一阶段的输入。

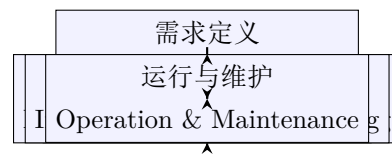


图 1: 瀑布模型的典型阶段结构

2.1.2 瀑布模型的特点

- 各阶段顺序相互依赖: 形成了软件生产流水线, 促进软件产业化和职业化发展。
- 每阶段生成文档并进行评审: 文档驱动 (Document-Driven), 软件过程可视化, 有利于质量保障和项目管理。
- 强调需求分析和设计: 推迟物理实施 (Deferring Physical Implementation), 有助于在早期发现和修正问题, 提高软件质量。

瀑布模型比较适合于**功能和性能需求明确**的软件项目的开发和维护, 如编译器、操作系统等。[p.20]

2.1.3 瀑布模型的不足

- 开发前期用户需求模糊不全面: 难以适应需求模糊的情况。
- 在开发过程中, 用户看不见系统: 得不到用户的反馈, 容易导致开发出的系统并不是用户真正需要的系统。
- 需求不确定或快速变化时返工代价大: 难以适用非线性顺序的开发活动。

[p.21]

2.2 原型模型 (*Prototype Model*)

2.2.1 模型概述

原型 (Prototype) 是一个可以实际运行的模型, 它在功能上可以看作是最终产品的一个子集, 展示了目标系统的关键功能。[p.23]

原型模型的核心思想是: **通过向用户提供可运行的原型来获取用户反馈**, 然后逐步求精完善原型, 最终得到目标系统。

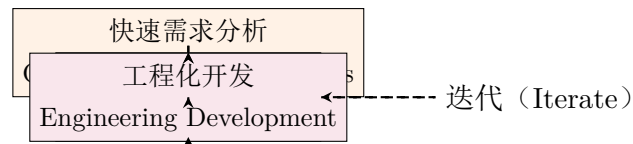


图 2: 原型模型的迭代开发过程

2.2.2 快速原型模型的特点

- 通过向用户提供原型获取用户的反馈：用户使用原型能够真正反映用户的需求（百闻不如一见）。
- 采用逐步求精方法完善原型：避免冗长的开发过程。
- 快速开发原型：便于快速响应用户意见。
- 不断迭代改进：更符合人们开发软件的习惯。

原型模型比较适合于需求模糊或不确定的软件项目的开发和维护。[p.24]

2.2.3 原型模型的不足

- 不宜利用原型系统作为最终产品：原型可能对软件系统异常等问题考虑较少，原型主要用于获取需求。有些原型也可以融入最终系统，但需要严格控制。
- 原型模型的“快速”特点对最终系统不适用：原型开发往往尽量使用能缩短开发周期的语言和工具，而最终系统修改可能不像原型修改一样快。

重要提醒：采用原型模型开发系统，用户和开发者必须达成一致——原型被建造仅仅是用来定义需求，之后便部分或全部抛弃，最终的软件要在充分考虑了性能和可维护性等质量方面之后才被开发。[p.25]

2.3 快速应用开发模型 (*RAD Model — Rapid Application Development*)

2.3.1 模型概述

RAD 模型也是一种线性的软件开发模型，但是这种模型特别强调采用极短的开发周期（比如 2-3 个月）。[p.27]

RAD 模型将开发过程分为五个顺序阶段：



图 3: RAD 模型的五个阶段

各阶段说明：[p.28]

- (1) 业务建模 (Business Modeling)：弄清业务活动中的信息流
- (2) 数据建模 (Data Modeling)：精化业务建模结果
- (3) 处理建模 (Process Modeling)：依据数据建模结果，创建处理描述
- (4) 应用生成 (Application Generation)：组件复用与开发

(5) **测试 (Testing)**: 测试新的组件及所有接口

RAD 模型主要用于**信息系统应用软件**的开发。[p.28]

2.3.2 RAD 模型的特点

- 顺序开发（如同瀑布模型），但强调极短的开发周期（2-3 个月）
- 通过组件复用加速开发过程
- 避免冗长的开发过程

2.3.3 RAD 模型的不足

- 需要足够的人力以创建足够的 RAD 小组：RAD 需要若干 RAD 小组并行工作。
- 技术风险很高的情况不适合采用：当新软件要求与已存在的程序有高可互操作性时，或系统难以被适当地划分为若干独立组件时。
- 开发者和用户需要做到在很短的时间内完成系统开发：对人员和协作要求很高。

[p.29]

2.4 增量模型 (Incremental Model)

2.4.1 模型概述

演化模型都是**迭代增量式**的，使得软件工程师能够开发出越来越完善的软件版本。**增量模型 (Incremental Model)** 由若干轮开发过程组成，每一轮都是在上一轮基础上进行。[p.31]

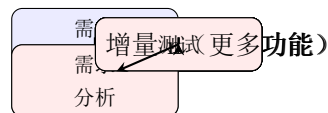


图 4: 增量模型的多轮迭代开发结构

2.4.2 增量模型的特点

- 结合了瀑布模型的直线式特点和快速原型化模型的迭代思想：增量模型中的每一轮开发过程是直线式的（像瀑布），但每一轮都是在上一轮基础上迭代（像原型）。
- 增量模型的每一轮都是**产品**（而非原型模型每一轮得到的是原型）：
 - 不需要大的资金支出，投资回报随功能渐增而渐增
 - 能够减少全新软件产品对用户带来的影响
 - 用户能及早使用及早发现问题
 - 能够有计划地管理技术风险
- 可以根据需要补充人员：灵活的项目资源配置。

[p.32]

2.4.3 增量模型的不足

- **对设计水平要求较高**：如果产品整体结构设计不当，则难以为其增加新的增量。这要求在开始时就有良好的架构设计（**Open Architecture**）。
- **由于采用增量开发，故难于进行彻底的测试**：有些增量开发在后期，回归测试的复杂度随增量增加而增加。

[p.33]

2.5 螺旋模型 (*Spiral Model*)

2.5.1 模型概述

通过使用原型或其他类似的手段以降低风险的思想，是**螺旋模型（Spiral Model）**中蕴涵的基本思想。最早由巴利·玻姆（Barry Boehm）于 1988 年提出。[p.35]

螺旋模型从中心开始，沿螺旋线向外扩展，每一圈代表一个迭代周期，包含四个主要阶段。

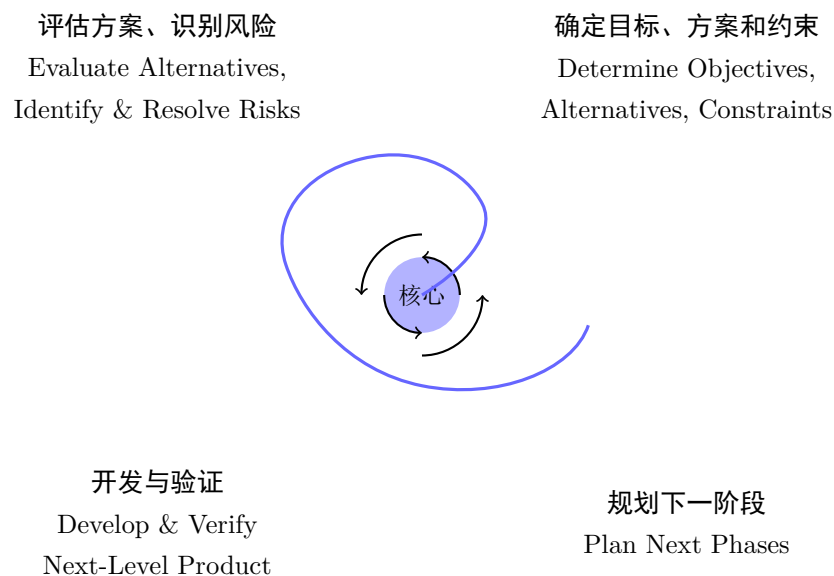


图 5: 螺旋模型概念图（四个象限代表四个主要活动）

螺旋模型的每一轮（螺旋的一圈）包含以下活动：

- (1) **确定目标（Determine Objectives）**：定义本轮迭代的目标、备选方案和约束条件
- (2) **评估风险（Identify & Resolve Risks）**：评估备选方案，识别并解决风险，必要时通过原型排除风险
- (3) **开发与验证（Develop & Verify）**：进行本轮的开发工作并验证结果
- (4) **规划下一阶段（Plan Next Phase）**：评审本轮成果，规划下一轮迭代

[p.36]

2.5.2 螺旋模型的特点

- **螺旋模型是风险驱动的（Risk-Driven）**：风险分析使得用户和开发者能够更好地理解和对待每一个阶段的风险，风险分析的目的是“去风险”。

- 结合了诸多模型的特点：
 - 保持了传统生命周期模型中系统的阶段性方法（像瀑布）
 - 通过建造原型来排除风险（像原型模型）
 - 自内向外、逐步延伸，体现了增量模型的特点（像增量）
- 螺旋模型适合于大型软件的开发：尤其是内部创新性项目。

[p.36]

2.5.3 螺旋模型的不足

- 要求软件开发人员擅长风险分析：重大风险没有被开发人员识别，或不是风险当成风险。
- 风险分析会导致项目终止而终止合同：出现违约诉讼，一般适合内部大系统开发。
- 风险分析会增加项目成本：对于小项目，风险分析的成本可能与整个项目的成本相当。

[p.37]

2.6 统一过程模型 (*Unified Process* — *UP* / *RUP*)

2.6.1 模型概述

统一过程（**Unified Process**，简称 **UP**）模型是 Rational 软件公司（现已被 IBM 收购）开发的一个基于统一建模语言（**Unified Modeling Language**，简称 **UML**）的迭代增量式软件开发模型。**Rational 统一过程**（**Rational Unified Process**，简称 **RUP**）是统一过程模型的一个商业版本。[p.39]

2.6.2 四个开发阶段

RUP 将软件开发过程划分为四个阶段，每个阶段包含若干迭代：

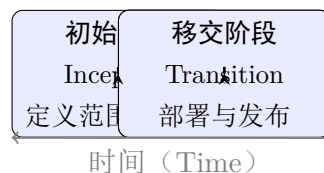


图 6: RUP 的四个开发阶段

各阶段说明：[p.40-43]

- (1) **初始阶段 (Inception)**：定义项目范围与目标，进行可行性分析，识别关键用例，评估风险，制定初步的项目计划。
- (2) **细化阶段 (Elaboration)**：进行详细的需求分析，建立体系结构基线，制定详细的开发计划，解决主要技术风险。
- (3) **构造阶段 (Construction)**：通过多次迭代增量构建系统，实现所有功能和需求，进行单元测试和集成测试。
- (4) **移交阶段 (Transition)**：将系统部署到用户环境中，进行验收测试、用户培训、文档编写和最终发布。

2.6.3 核心工作流 (Core Workflows)

RUP 定义了 9 个核心工作流 (6 个过程工作流 + 3 个支持工作流): [p.44]

类型	工作流名称	英文
过程工作流	业务建模	Business Modeling
	需求	Requirements
	分析与设计	Analysis & Design
	实现	Implementation
	测试	Test
	部署	Deployment
支持工作流	配置与变更管理	Configuration & Change Management
	项目管理	Project Management
	环境	Environment

表 1: RUP 的 9 个核心工作流

2.6.4 RUP 的三大特点

- **用例驱动 (Use-Case Driven)**: 用例是指系统外部参与者完成的一系列动作, 用例确定了开发目标并驱动每次迭代的工作。
- **以体系结构为中心 (Architecture-Centric)**: 体系结构提供了一种结构来指导迭代过程中的工作, 有助于项目涉众对新系统有一个宏观的认识, 有助于软件重用。
- **迭代增量式地进行软件开发 (Iterative and Incremental)**: 有助于适应需求的变化, 能使项目计划更加灵活。

[p.45]

2.6.5 RUP 的不足

- **太复杂**: 依据项目需要进行裁剪, 但统一过程模型并未给出具体的裁剪实施方法。
- **太重 (Heavyweight)**: 各阶段必须开发大量的制品, 开发的大量制品为开发过程服务, 不适合资源有限且面临时间压力的软件项目。

[p.46]

3 传统过程模型对比 (*Comparison of Traditional Models*)

表 2: 六种传统软件过程模型对比

模型	核心特点	优点	缺点	适用场景
瀑布模型 (<i>Waterfall</i>)	顺序执行, 阶段递进; 文档驱动	过程可视化, 便于管理, 质量有保证	需求必须事先明确, 用户反馈滞后, 返工代价大	需求明确的系统软件 (如编译器、操作系统)
原型模型 (<i>Prototype</i>)	快速构建可运行原型, 获取用户反馈	能发现真实需求, 用户参与度高, 逐步求精	原型可能被误用为最终产品, 最终系统修改不快速	需求不明确或模糊的项目
RAD 模型	极短开发周期 (2-3 月), 组件复用, 顺序开发	开发速度快, 组件复用降低成本	需要大量人力, 不适用高技术风险或难以分解的系统	信息系统应用软件
增量模型 (<i>Incremental</i>)	多轮直线式开发, 每轮出产品增量	投资回报渐进, 风险可管, 用户早使用	需要良好架构设计, 整体测试困难	需求基本明确但可分阶段交付的项目
螺旋模型 (<i>Spiral</i>)	风险驱动, 结合瀑布/原型/增量, 四象限迭代	风险管理最佳, 适合大型项目	需要风险分析专家, 增加项目成本, 小项目不划算	大型软件、内部创新项目
统一过程 (<i>RUP</i>)	用例驱动, 体系结构为中心, 迭代增量	过程规整, 体系结构清晰, 重用性好	太复杂太重, 需要大量制品和文档, 裁剪困难	大型企业级软件项目

4 敏捷软件开发 (*Agile Software Development*)

4.1 敏捷运动的背景

软件开发常常不能按时交付,也不能交付价值。这种失败具有巨大的经济和人力影响。[p.51]
传统的基于计划或描述的软件开发模型(如瀑布模型),不仅费用昂贵,而且由于外部环境的变化造成返工非常普遍。[p.52]

受到敏捷制造(**Agile Manufacturing**)的启发,软件工程领域提出了敏捷价值观和原则,并在迭代增量式软件开发的基础上,总结出各种敏捷实践。[p.52]

4.2 敏捷联盟与敏捷宣言 (*Agile Alliance & Agile Manifesto*)

2001 年 2 月,肯特·贝克(**Kent Beck**)和其他 16 位软件产业界专家学者,成立了敏捷联盟(**Agile Alliance**),共同签署了敏捷联盟宣言(**The Manifesto of the Agile Alliance**)。[p.54]

敏捷价值观把人的因素放在第一位,强调以下几个方面的能力: [p.54]

- 基本能力: 应该具有关于过程的知识 and 技能
- 共同目标: 在承诺的时间和资源约束下, 向客户提交可运行的软件增量
- 精诚合作: 沟通、反馈
- 决策能力: 自主决策权
- 模糊问题的解决能力
- 相互信任和尊重
- 自组织 (Self-Organization)

4.2.1 敏捷的定义

什么是敏捷? [p.55]

- Effective (rapid and adaptive) response to change ——对变化做出有效(快速且自适应)的响应
- Effective communication among all stakeholders ——所有利益相关者之间的有效沟通
- Drawing the customer onto the team ——把客户融入团队
- Organizing a team so that it is in control of the work performed ——组织一个能够掌控其工作的团队

结果是: **Rapid, incremental delivery of software** ——快速、增量地交付软件。[p.55]

4.3 12 条敏捷原则

敏捷联盟提出了 **12 条敏捷原则**, 是敏捷实践区别于传统过程模型的重要特征。[p.57]

编号	维度	原则内容
1	物 (价值)	最优先要做的是通过尽早地、持续地交付有价值的软件来使客户满意。
2	物 (需求)	即使在开发后期，也欢迎需求改变。敏捷过程利用变化来为客户创造竞争优势。
3	物 (交付)	经常性地交付可以工作的软件，交付的间隔可以从几个星期到几个月，时间越短越好。
4	人 (工作)	在整个项目开发期间，业务人员和开发人员必须天天在一起工作。
5	人 (个人)	围绕有积极性的个人构建项目团队。提供所需的环境和支持，并信任他们能够完成工作。
6	人 (交流)	在团队内部，最有效并富有效率的信息传递方法是面对面的交流。
7	物 (进度)	可运行的软件是首要的进度度量标准。
8	人 (速度)	敏捷过程提倡可持续的开发速度。责任人、开发者和用户应该能够保持长期、稳定的速度。
9	人 (能力)	持续关注优秀的技能和好的设计，增强敏捷能力。
10	物 (简单)	简单（是不必做的工作最大化的艺术）是必要的。
11	人 (团队)	最好的架构、需求和设计出自于自组织的团队。
12	人 (反省)	每隔一段时间，团队应反省如何才能有效地工作，并相应地调整自身的行为。

表 3: 12 条敏捷原则（简表）

4.3.1 敏捷原则的分类理解

从组织方面和技术方面两个维度来理解 12 条原则：[p.58]

组织方面	技术方面
客户：使客户满意	需求：需求变化
客户：为客户创造竞争优势	设计：好的设计
个人：有积极性的个人	实现：简单
个人：优秀技能	交付：尽早持续交付
团队：欢迎需求改变	交付：交付时间短
团队：面对面交流	交付：可工作的软件
团队：自组织团队	交付：有价值的软件
团队：团队反省	
团队：客户参与（天天在一起）	
进度：用可运行的软件衡量	
进度：可持续的开发速度	

表 4: 敏捷原则的组织与技术维度分类

5 极限编程 (*Extreme Programming — XP*)

5.1 XP 概述

极限编程 (Extreme Programming, 简称 XP) 是由肯特·贝克 (Kent Beck)、沃德·库宁汉姆 (Ward Cunningham) 和罗恩·杰弗里斯 (Ron Jeffries) 等人通过整理优秀团队的共同之处而提出的敏捷过程。[p.60]

所谓极限, Kent Beck 认为是: 尽力而为, 然后处理其结果。

极限编程专注于编程技术、清晰沟通和团队协作, 只需做能够为客户创造价值的事情, 是一组确保项目开发成功的规则, 适用于任何规模的团队, 适合模糊或快速变化的需求。[p.60]

5.2 XP 与瀑布模型及迭代模型的区别

极限编程与传统瀑布模型和一般迭代模型的主要区别在于: [p.61]

- 瀑布模型: 各阶段 (需求、设计、编码、测试) 按顺序一次性完成
- 迭代模型: 各阶段在多轮迭代中按顺序完成
- 极限编程: 各阶段在多轮迭代中同时进行, 持续集成、持续反馈

5.3 XP 的五大价值观 (*Five XP Values*)

极限编程包含五个指导软件开发的价值观: [p.62]

- (1) 交流 (Communication): 在创造团队意识和高效合作意识方面, 交流是最为有效的手段。
- (2) 简单 (Simplicity): 简单化会使得集中精力满足当前的需要、交流的内容大为减少、保持代码简洁、去掉不需要或暂缓的需求等。
- (3) 反馈 (Feedback): 反馈用来修正错误, 有益于获取简单的解决方案。
- (4) 勇气 (Courage): 勇气是敏捷软件开发必备的基本要素。需要勇气保持简单; 需要勇气进行交流; 需要勇气面对错误修正。
- (5) 尊重 (Respect): 尊重体现在彼此关心、相互理解。

5.4 XP 的实践原则

极限编程原则是一组领域特定的、用于搜寻与极限编程价值观相协调的实践指南。一方面, 极限编程原则可以用来理解极限编程实践, 另一方面也可以依据这些原则补充新的实践。[p.63]
通过在软件开发过程中运用极限编程实践来实现极限编程的价值。[p.64-65]

5.5 XP 核心实践体系

极限编程过程将各种极限编程实践进行有效地结合。[p.66]

XP 的核心实践包括:

类别	实践	说明
规划	用户故事	描述用户可见的任务，可估算、可测试、可完成、可沟通
	故事点估算	扑克法、三角测量法估算工作量
	计划发布	依据业务价值和时间为发布选择故事
开发	结对编程	两人共用一台电脑，一人编码一人评审
	测试驱动开发	先写测试，再写代码使测试通过
	重构	不改变行为的前提下改进代码结构
	持续集成	每天多次合并代码，自动化测试与构建
	代码集体所有	任何成员可修改任何代码
	简单设计	最简单可行的设计方案
	编码标准	统一的编码规范与约定
团队	整支团队	跨职能团队在同一空间协作
	可持续节奏	保持稳定、可持续的开发速度
	系统隐喻	用比喻来理解系统架构

表 5: XP 核心实践

5.6 XP 开发过程详解

5.6.1 规划阶段（Planning Phase）

极限编程从**架构探索**和**初始用户故事**开始，开发人员通过与客户交流，尽量确定出真正重要的系统将要如何工作的故事。[p.67]

规划阶段应该完成以下工作：[p.67]

- 编写足够多的用户故事，以定义成功的产品
- 做必须的探索实验，以便对用户故事进行估算
- 估算每一个故事实施的时间
- 依据业务价值和时间为发布选择故事

系统隐喻（System Metaphor） 在用户故事和对潜在解决方案探索的基础上，开发人员可以决定系统采用什么样的架构。为了便于大家理解探索得到的架构，开发人员往往会构建一个**系统隐喻**。[p.68]

例如：

- 基于本体的信息检索系统 → 隐喻：程序工作就像一群蜜蜂，外出寻找花粉，并将花粉带回蜂巢。
- 文本输出缓冲系统 → 需要思考出一个好的系统隐喻来描述缓冲机制。

系统隐喻促进了团队对系统架构的**共同理解（Common Understanding）**。[p.68-69]

用户故事 (User Story) 用户故事描述了从用户角度来看系统认为有价值的任务。用户故事应该满足以下要求: [p.70]

- (1) **可估算**: 故事不能太大, 且开发人员具备开发技术
- (2) **可测试**: 可以验证对错 (例如 “用户界面友好” 就不可测试)
- (3) **可完成**: 一个迭代周期 (如 2 周) 能完成
- (4) **可沟通**: 一次不需要引入太多细节, 否则不便于沟通, 细节在迭代时产生

用户故事一般由两部分组成: [p.71]

- 书面卡片: 由客户描述的软件输入、输出、特征以及功能等组成
- 对话记录: 记录完成故事编写时发生在开发人员和客户之间的一系列对话, 供后续阶段使用

用户故事的常见问题 : [p.71]

- “生成报告以便可以理解哪些部门需要更多资源” —— 太大太模糊
- “作为学生, 需要一个运行良好的课程网上课程注册功能, 以便随时选课” —— 范围太大
- “作为管理人员, 需要一个决策支持功能, 以便辅助决策” —— 无法测试

用户故事点估算 用户故事点 (Story Point) 是团队定义开发时间的模糊单位, 可衡量开发故事的工作量。[p.72]

估算方法:

- **扑克法 (Planning Poker)**:
 - 每位成员得到一副带数字 0、0.5、1、2、3、5、8、... 的卡片
 - 用户讲解用户故事, 团队成员质疑, 用户解答
 - 各成员估算故事点 (可选择基准故事, 故事点数假定为 1, 各成员将故事与基准故事比较)
 - 处理估算结果: 将结果求平均, 取最接近的数字作为故事点数; 如果结果不同, 讨论后再次估算, 直到结果一致 (不超过三次)
- **三角测量法 (Triangulation)**: 将被估算故事与其他多个故事的相对关系做比较。

开发速度 (Velocity) = 总故事点 / 总完成时间。[p.72]

5.6.2 迭代阶段 (Iteration Phase)

在规划阶段, 客户和开发人员就发布计划达成一致。下一阶段便需要通过**迭代方式增量式**地完成发布计划, 也称为**构建迭代阶段**。[p.73]

迭代周期管理 即使没有在本次迭代周期完成所有需要完成的用户故事, 迭代也要在先前指定的日期前结束。[p.74]

客户和开发人员会依据已完成的用户故事的估算值, 计算本次迭代的**项目速度**, 即完成的用户故事的点数。项目速度会被用于计算下一次迭代的周期或用户故事点数。[p.74]

制定迭代计划

- 将用户故事分解成任务:

- 一个任务是指一个开发人员能够在 4-16 小时之内能够实现的功能
- 开发人员需要在客户的帮助下，通过详细分析，尽可能完全地列出所有的任务
- 开发人员逐个签订他们想要实现的任务：
 - 开发人员可以签订任意类型的任务
 - 依据最近一次迭代中所完成的任务点数来确定本次可以签订的任务点数
 - 任务的选择要一直到所有任务都被分配出去，或者所有的开发人员都已签满
 - 如果还有任务未分配，则开发人员协商，或要求客户去掉一些任务
 - 如果所有任务均已分配但仍有开发人员还能签订任务，则向客户要求更多的用户故事

[p.75]

开发：按照迭代计划进行代码开发。[p.76]

站立式会议（Stand-up Meeting）

- 团队集中讨论当前的工作，对具体问题寻求解决建议
- 一般持续很短的时间，应回答三个问题：
 1. 自昨天开始已经做了什么？
 2. 从现在开始你将做什么？
 3. 阻碍迭代目标的有什么？有没有未完成的事情？
- 目的是相互交流学习，了解项目的进度

[p.77]

代码集体所有（Collective Code Ownership） 任何团队成员都可以修改任何部分的代码，增强团队协作和知识共享。[p.78]

测试驱动开发（Test-Driven Development, TDD）

- 在开发产品代码之前，先编写测试代码，然后只编写使测试通过的产品代码
- 测试代码的三个作用：[p.79]
 1. 验证（Validation）：确保产品正确
 2. 设计（Design）：接口设计等（功能实现和功能使用分离）
 3. 文档（Documentation）：告诉其他人员如何使用代码
- TDD 过程：[p.79]
 1. 明确当前产品要完成的功能
 2. 编写测试代码
 3. 编写产品代码，使测试通过
 4. 必要时重构代码
 5. 重复上述步骤，直到所有测试通过

结对编程（Pair Programming） 所谓结对编程，是指两个人一起在一台电脑上完成代码的编写工作。一个为主写代码（Driver），另一个则对代码进行评审（Navigator/Observer）。[p.80]

结对编程的好处：

- 更早发现错误
- 有利于团队能力提升
- 提高编程效率
- 有利于实现代码共有
- 有助于增进团队内部凝聚力

重构 (Refactoring) 所谓重构, 是在不改变代码行为的前提下, 对代码内部结构进行渐进式改进的过程。[p.81]

确保行为不变的方法就是通过测试。重构的作用:

- 使代码永远保持活力
- 重构是一种代码清洗
- 重构是应对变化的方法

持续集成 (Continuous Integration) 将开发者的工作副本每天多次合并到主本的活动。[p.82]

持续集成完成以下工作:

- 单元测试 (Unit Testing)
- 代码检测 (Code Inspection)
- 编译构建 (Build)
- 甚至包括验收测试、自动部署等

持续集成的优势:

- (1) 提高了工作效率——将重复劳动自动化
- (2) 便于更快地修复错误——早集成, 早发现
- (3) 更快交付成果——减少手工错误, 减少各环节等待时间

常用持续集成工具: Jenkins、CruiseControl 等。[p.82]

5.6.3 测试阶段 (Testing Phase)

XP 测试阶段的关键要点: [p.83]

- **单元测试 (Unit Test):** 自动化测试
- **验收测试 (Acceptance Test):** 黑盒测试
- 所有测试在编码之前设计和创建
- 所有单元测试必须 100% 通过
- 客户负责功能测试 (验收测试)
- 每个用户故事被转化为若干个测试
- 为使系统具有可测试性, 需要在系统架构层面对系统解耦

5.6.4 发布阶段 (Release Phase)

极限编程过程强调小规模、频繁地发布代码和测试。[p.84]

- 小型发布有利于尽早为客户提供业务价值, 使客户增加信心
- 客户通过使用小型发布的软件系统, 能够获取更多的其他需求, 发现系统存在的缺陷

- 通过反馈将更有力地指导项目成功

在发布阶段需要撰写的文档包括：

- **系统文档：**为理解系统提供总览信息（系统技术架构、高层次系统需求、关键设计决策总结、重要的设计模型等）
- **系统操作文档：**描述系统涉及的依赖关系、与其他系统交互的特性、预期系统负载等
- **系统支持文档：**描述解决问题时的参考信息、疑难问题的上报流程、维护团队的联系列表等
- **用户文档：**参考手册、用户指南、支持指南及培训资料等

6 Scrum 敏捷软件开发过程

6.1 Scrum 概述

在敏捷软件开发中，**Scrum** 是一种迭代增量式软件开发过程，就像橄榄球赛的争球过程：快速、自组织和自适应性。[p.86]

Scrum 强调**管理**，而极限编程强调**实践**，两者具有很好的互补性。[p.95]

6.2 Scrum 团队的三种角色

Scrum 团队有三种核心角色：[p.87]

角色	英文	职责
产品负责人	Product Owner	定义和维护“产品待办事项表（Product Backlog）”，负责最大化产品以及 Scrum 团队的工作价值，代表利益相关者的利益。
Scrum 主管	Scrum Master	确保 Scrum 团队遵循 Scrum 理论、实践和规则，通过指导和引导，使 Scrum 团队更加高效地创建高质量的产品。
开发团队	Development Team	负责在每个冲刺（Sprint）结束时交付潜在可发布的“已完成”产品增量。只有开发团队的成员才能交付产品增量。

表 6: Scrum 团队的三种角色

6.3 开发团队的特点

团队的大小足够小，以保证灵活性，同时应能完成有意义的任务，一般是 7 ± 2 人。[p.88]

开发团队有以下几个特点：[p.88]

- 成员是**自组织的**，没有人（即使是 Scrum 主管都不可以）告诉开发团队如何把产品待办事项表变成潜在可发布的功能
- 开发团队是**跨功能的**，团队作为一个整体拥有创造产品增量所需要的全部技能
- Scrum 不认可开发团队成员的头衔，无论承担哪种工作他们都是开发者。此规则无一例外
- 开发团队中的每个成员可以有特长和专注领域，但是责任归属于整个开发团队
- 开发团队不包含如测试或业务分析等负责特定领域的子团队

6.4 PDCA 循环（戴明环）与冲刺的关系

PDCA 循环（Plan-Do-Check-Act）最初由 Walter A. Shewhart 于 1930 年提出，后经 W. Edwards Deming 于 1950 年推广。[p.89]

与 Scrum 冲刺的区别：

- PDCA 循环并没有严格的时长规定，Scrum 的冲刺有固定时长
- PDCA 循环并没有规定团队大小，Scrum 团队建议 7 ± 2 人

6.5 Scrum 制品 (*Scrum Artifacts*)

Scrum 软件开发过程产生的制品除可工作的软件外，主要有四种：[p.90]

6.5.1 产品待办事项表 (*Product Backlog*)

- 估算点数的单位是故事点个数，是一个模糊单位
- 好的产品待办事项表有 **DEEP** 四个特征：
 - Detailed appropriately ——适当详尽的
 - Emergent ——涌现的（随项目进展不断涌现）
 - Estimated ——经过估算的
 - Prioritized ——已排定优先级的

[p.91]

6.5.2 冲刺待办事项表 (*Sprint Backlog*)

冲刺待办事项表定义了开发团队把产品待办事项表条目转换成“完成”的增量所需要执行的任务。[p.92]

- 估算时间的单位是小时数
- 估算时间每天都更新

6.5.3 冲刺燃尽图 (*Sprint Burndown Chart*)

- 剩余工作量的单位是人-天数，也可以用人-时数
- 工作日以每周工作 5 天
- 燃尽图直观展示冲刺过程中剩余工作量随时间的变化趋势

[p.93]

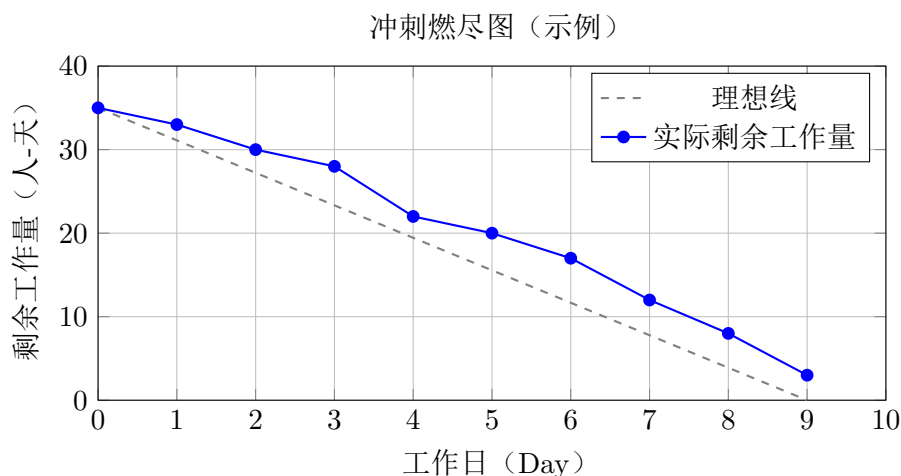


图 7: 冲刺燃尽图示意

6.5.4 发布燃尽图 (Release Burndown Chart)

通过燃尽图，可以确定团队的工作情况，了解计划的执行情况，用于团队进度的跟踪，此外还可以分析团队有哪些可以改进。[p.94]

6.6 Scrum 会议 (*Scrum Meetings*)

Scrum 包含四类核心会议：[p.95]

6.6.1 冲刺计划会 (*Sprint Planning Meeting*)

- 为冲刺做准备的会议，主要确定冲刺要做什么和怎么做，时间大概是几个小时
- 开发团队和产品负责人通过共同讨论，理解产品负责人需要什么和为什么需要
- 由开发团队自己确定本次冲刺应该完成的产品待办事项表中的条目
- 开发团队对要在本次冲刺中实现的条目进行计划、分析和设计，将每个条目分解成细粒度的任务，形成冲刺待办事项表和冲刺目标

[p.96]

如何确定本次产品待办事项（示例） 第一步：计算团队的开发速度

- 已完成的故事点数 24 / 工作总天数 53 = **0.45 故事点/天**
- 未完成的产品待办事项不纳入计算

[p.97]

第二步：计算本次可以完成的故事点数

- 团队开发速度 0.45 故事点/天 × 团队可以工作的总天数 49 = 22.05 故事点
- 选取最接近的整数为 **22 个故事点**
- 据此选取产品待办事项编码为：1, 2, 3, 4

[p.98]

6.6.2 每日站立会议 (*Daily Stand-up*)

- 要求每个成员都参加，时间不超过 **15 分钟**
- 所有成员必须回答三个问题：
 1. 上次站立会议后做了哪些工作？
 2. 遇到了哪些问题？
 3. 下次站立会议之前计划做什么？
- Scrum 主管负责帮助团队解决遇到的问题
- 会后会有一个或多个并行的跟进会议（不要求所有人都参加），主要针对站立会议收集的信息与相关成员作进一步的沟通

[p.99]

6.6.3 冲刺评审会议 (*Sprint Review Meeting*)

- 对功能性的产品增量进行审视和调整，时间不超过 4 小时（小时数等于本次冲刺周期的周数）

- 在冲刺评审会中，真实用户和产品负责人检验和使用运行起来的软件
- 通过开发团队、产品负责人和其他涉众之间的交流，审视产品的进展，并针对问题进行调整

[p.100]

6.6.4 冲刺反思会 (*Sprint Retrospective*)

- 在冲刺评审会之后，针对流程和环境进行审视和调整
- 每位成员对本次冲刺的情况进行回顾，不仅对工作中存在的问题进行反思，而且也要讨论好的工作方式
- 每位成员要对其他成员的反思进行评价，表达各自的期望

[p.101]

7 其他敏捷方法 (*Other Agile Methods*)

7.1 敏捷方法的实际使用统计

有人统计，在行业实际使用的所有敏捷软件开发过程中：[p.103]

- 极限编程过程占 8%
- Scrum 过程占 49%
- 极限编程和 Scrum 结合过程占 22%
- 其他敏捷软件开发过程占 21%

可见 Scrum 是使用最广泛的敏捷方法，而 XP 与 Scrum 的结合也很常见。

7.2 精益软件开发 (*Lean Software Development*)

精益制造 (Lean Manufacturing)，通常简称为“精益”，是一种优先考虑资源有效利用的生产实践，只关注直接为最终客户贡献价值的活动，同时最大限度地减少浪费性支出。[p.104]

精益软件开发是借用精益制造原则形成的敏捷软件过程，强调在产品开发过程中消除浪费和官僚主义，鼓励通过短周期频繁构建进行学习，提升后期改变和快速迭代，通过反馈将变化吸收到产品中，而不是采用需求文档和严格的计划来推动软件开发。[p.104]

精益软件开发的七个核心原则：[p.104]

- 整体优化 (Optimize Whole)
- 消除损耗 (Eliminate Waste)
- 构建内在品质 (Build Quality In)
- 持续学习 (Learn Constantly)
- 快速交付 (Deliver Fast)
- 吸引每一个人 (Engage Everyone)
- 越来越好 (Keep Getting Better)

7.3 特征驱动软件开发 (*Feature-Driven Development — FDD*)

7.3.1 什么是特征

所谓**特征 (Feature)**，是指一个小的、具有客户价值的功能：[p.105]

- **特征的小**：该功能可以在两个星期之内完成
- **特征具有客户价值**：该功能可以映射到业务过程中某些活动的步骤
- 特征与极限编程中的用户故事相似，都是从客户的角度来理解软件需求

7.3.2 FDD 的八个最佳实践

- 领域对象建模 (Domain Object Modeling)
- 按特征开发 (Develop by Feature)
- 个体类 (代码) 拥有权 (Individual Class Ownership)
- 特征小组 (Feature Teams)
- 审查 (Inspections)

- 定期构造 (Regular Builds)
- 配置管理 (Configuration Management)
- 结果可视性报告 (Progress & Results Visibility)

[p.105]

7.3.3 FDD 的五个子过程

- 开发全局模型 (Develop Overall Model)
- 构造特征表 (Build Feature List)
- 依据特征制定计划 (Plan by Feature)
- 特征设计 (Design by Feature)
- 特征构造 (Build by Feature)

[p.105]

8 敏捷项目管理工具 (*Agile Project Management Tools*)

8.1 PingCode

PingCode 可能是国内最成熟、最标准的敏捷开发项目管理软件。[p.107]

适合十几人至几百人等不同规模的敏捷开发团队。PingCode 完整支持标准的 Scrum 敏捷开发流程、敏捷 Kanban 开发流程，以及规模化敏捷的管理。PingCode 还支持私有部署、二次定制开发和适配国产系统。[p.107]

8.2 Gitee 项目管理

Gitee，原名 OSC Git，于 2013 年正式更名并对外开放服务。Gitee 项目管理覆盖了研发管理全生命周期，支持瀑布、敏捷（Scrum、Kanban）等多种开发模型，与代码管理、CI/CD、测试管理等功能无缝集成，同时提供强大且灵活的自定义配置能力。[p.115]

Gitee 项目管理的主要功能模块：[p.116-124]

- **创建项目：**Gitee 以项目为管理单位，首先要先创建一个项目
- **成员管理：**添加项目成员，分配权限
- **需求和任务管理：**需求可以分解为若干任务
- **代码仓库：**每一个项目都有一个代码仓库，项目成员可以 fork 对应模板仓库到自己的项目下
- **代码迭代和版本号管理：**
 - 1 级编码代表一个项目迭代
 - 2 级编码代表一次需求增加
 - 3 级编码代表增加一个任务迭代
- **工程代码 Fork**
- **代码提交：**两种方式——网页版提交和 Git 命令行提交

9 本章核心概念总结

9.1 关键术语中英对照表

中文术语	英文术语	简要说明
软件过程	Software Process	相互有机联系的活动集合
软件过程模型	Software Process Model	活动/任务/子过程的有机结合
软件生存周期模型	SDLC Model	也称为软件开发模型
瀑布模型	Waterfall Model	顺序执行，文档驱动
原型模型	Prototype Model	快速获取用户反馈
快速应用开发	RAD — Rapid Application Development	极短开发周期
增量模型	Incremental Model	多轮直线式开发
螺旋模型	Spiral Model	风险驱动
统一过程	Unified Process (UP)	基于 UML 的迭代增量式模型
RUP	Rational Unified Process	UP 的商业版本
敏捷软件开发	Agile Software Development	快速、自适应、增量交付
敏捷宣言	Agile Manifesto	2001 年敏捷联盟签署
极限编程	Extreme Programming (XP)	专注编程技术和团队协作
测试驱动开发	Test-Driven Development (TDD)	先写测试再写代码
结对编程	Pair Programming	两人共用一台电脑编程
重构	Refactoring	不改变行为前提下改进代码结构
持续集成	Continuous Integration (CI)	每天多次合并代码
Scrum	Scrum	强调管理的敏捷过程
产品待办事项表	Product Backlog	产品功能优先列表
冲刺	Sprint	Scrum 的时间盒迭代
冲刺待办事项表	Sprint Backlog	冲刺内要完成的任务
燃尽图	Burndown Chart	剩余工作量可视化
用户故事	User Story	从用户角度描述有价值的任务
故事点	Story Point	工作量模糊估算单位
精益软件开发	Lean Software Development	借用精益制造原则
特征驱动开发	Feature-Driven Development (FDD)	按特征开发

9.2 传统模型与敏捷方法的对比

传统模型 (Traditional)	敏捷方法 (Agile)
基于计划 (Plan-Driven)	基于变化 (Change-Driven)
文档驱动	代码驱动 (可运行软件)
阶段性、顺序式开发	迭代增量式开发
需求在前期确定	需求随项目演进
用户早期参与不足	客户始终融入团队
适合稳定需求	适合快速变化的需求
过程重、制品多	过程轻、简单化
风险在前期分析	持续风险管理
顺序测试	测试驱动开发 (TDD)
大型团队	小团队 (7 ± 2 人)

表 8: 传统模型与敏捷方法的对比